

Artificial Intelligence and Machine Learning Lab

Enrolment No: _____

Division: _____

Roll No: _____

Name: _____

Experiment No.: 05

Title: Develop a Bayesian classifier.

Theory:

Bayes Theorem

The formula is:

$$P(A|B) = P(A) P(B|A)/P(B)$$

Which tells us:	how often A happens <i>given that B happens</i> , written P(A B) ,
When we know:	how often B happens <i>given that A happens</i> , written P(B A)

and how likely A is on its own, written **P(A)**

and how likely B is on its own, written **P(B)**

Bayes Classifier with example

In machine learning, naïve Bayes classifiers are a family of simple "probabilistic classifiers" based on applying Bayes' theorem with strong (naïve) independence assumptions between the features. They are among the simplest Bayesian network models.[1] But they could be coupled with Kernel density estimation and achieve higher accuracy levels.

Naive Bayes is a simple technique for constructing classifiers: models that assign class labels to problem instances, represented as vectors of feature values, where the class labels are drawn from some finite set. There is not a single algorithm for training such classifiers, but a family of algorithms

Artificial Intelligence and Machine Learning Lab

based on a common principle: all naive Bayes classifiers assume that the value of a

particular feature is independent of the value of any other feature, given the class variable. For example, a fruit may be considered to be an apple if it is red, round, and about 10 cm in diameter. A naive Bayes classifier considers each of these features to contribute independently to the probability that this fruit is an apple, regardless of any possible correlations between the color, roundness, and diameter features.

**Pseudo
code or
Program**

```
# Import necessary libraries
```

```
import joblib
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.metrics import classification_report, accuracy_score
```

```
# Sample dataset for text classification (You can replace this with any dataset  
you have)
```

```
# For example, let's assume we have a dataset of emails that are labeled as 'spam'  
or 'ham'
```

```
data = {
```

```
    'text': [
```

```
        "Free money now", "Win a free iPhone", "Hello, how are you?",
```

```
        "Meeting tomorrow at 10 AM", "Claim your prize", "Please review my  
report",
```

```
"Urgent offer, buy now", "Are we still on for the meeting?", "Get paid for surveys",
```

```
"Let's grab coffee today"
```

```
],
```

```
'label': ['spam', 'spam', 'ham', 'ham', 'spam', 'ham', 'spam', 'ham', 'spam', 'ham']
```

```
}
```

```
# Convert the dataset into a pandas DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Step 1: Data Preprocessing (Splitting the data into training and testing sets)
```

```
X = df['text'] # Features (text data)
```

```
y = df['label'] # Target labels (spam or ham)
```

```
# Split data into training (80%) and testing (20%)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

```
# Step 2: Feature Extraction (Converting text to numerical features using TF-IDF)
```

```
vectorizer = TfidfVectorizer(stop_words='english') # Remove English  
stopwords during vectorization
```

```
# Fit and transform the training data
```

```
X_train_tfidf = vectorizer.fit_transform(X_train)
```

```
# Transform the test data using the same vectorizer (important to use the same  
one)
```

```
X_test_tfidf = vectorizer.transform(X_test)

# Step 3: Train Naive Bayes Classifier
model = MultinomialNB()
model.fit(X_train_tfidf, y_train)

# Step 4: Model Evaluation (Make predictions and evaluate)
y_pred = model.predict(X_test_tfidf)

# Evaluate the performance of the model
print("Classification Report:\n", classification_report(y_test, y_pred))

print("Accuracy Score:", accuracy_score(y_test, y_pred))

# Step 5: Saving the trained model using joblib for later deployment
joblib.dump(model, 'naive_bayes_model.pkl')
joblib.dump(vectorizer, 'tfidf_vectorizer.pkl')

print("Model and vectorizer saved successfully!")

# Step 6: Deploying the model (Using the saved model to make predictions)
# Load the saved model and vectorizer
loaded_model = joblib.load('naive_bayes_model.pkl')
loaded_vectorizer = joblib.load('tfidf_vectorizer.pkl')

# Make prediction on new unseen data
new_text = ["Get your free gift now!", "Let's catch up soon"]
```

```
new_text_tfidf = loaded_vectorizer.transform(new_text) # Transform the new
text using the saved vectorizer
```

```
predictions = loaded_model.predict(new_text_tfidf)
```

```
# Print the predictions for the new text
```

```
for text, pred in zip(new_text, predictions):
```

```
    print(f"Text: {text}\nPredicted Label: {pred}\n")
```

```
2)
```

```
import pandas as pd
```

```
from sklearn.naive_bayes import GaussianNB
```

```
df_=pd.read_csv('IRIS.csv')
```

```
df_=df_.dropna(axis=1,how='any')
```

```
from sklearn.preprocessing import StandardScaler from
sklearn.model_selection
```

```
import train_test_split from sklearn.neural_network
```

```
import MLPClassifier from sklearn.preprocessing
```

```
import LabelEncoder
```

```
le = LabelEncoder()
```

```
le.fit(["Iris-setosa", "Iris-versicolor", "Iris-viginica"])
```

```
le.transform(["Iris-setosa", "Iris-versicolor", "Iris-viginica"])
```

```
x=df_.iloc[:,0:3]
```

```
obj=StandardScaler()
```

```
x_=obj.fit_transform(x)
```



Artificial Intelligence and Machine Learning Lab

```
y=df['species']  
  
X_train, X_test, y_train, y_test = train_test_split(x_, y, test_size=0.4,random_state=42)  
  
X_validate, X_test1, y_validate, y_test1=train_test_split(X_test,y_test,test_size=0.5,random_state=42)  
  
bayes=GaussianNB(priors=None)  
  
bayes.fit(X_train,y_train)  
  
print(bayes.score(X_validate,y_validate))  
print(bayes.score(X_test1,y_test1))
```

Outputs

Artificial Intelligence and Machine Learning Lab

(Students can attach extra pages if necessary)

Exercise for Practice

Q1. How can Bayes classifier can be used for categorical features? What if some features are numerical?

Ans:

Q2. What are advantages and disadvantages of Naive Bayes Algorithm?

Ans

Q3 How is Bayes' Theorem relevant to the field of artificial intelligence?

Ans

Q4. What are the Applications of Naive Bayes Classifier?

Ans



Artificial Intelligence and Machine Learning Lab

Q5	Discuss a situation where the naive Bayes' approach would not be appropriate to use.
Ans	
Conclusion	Thus, we have successfully completed the implementation of Naïve Bayes Gaussian Classifier.

Date of Submission:	
Marks out of 10	Name and Sign of Subject Teacher
Remark:	

